# Dory.md — Person 3

*Frontend*

*React + TypeScript · Tailwind · Recharts · Time Machine slider · Discovery card · Vercel deploy*

| Your role | **Frontend Engineer — you own everything in /frontend/. You build the UI the judges actually see and interact with. You ship the demo's signature moment (the Time Machine slider).** |
|---|---|
| What you ship | A React + TypeScript + Tailwind + Recharts SPA. Six screens: dashboard with category treemap, chunk grid, search results, Time Machine slider, Discovery notification card, quiz mode. Connects to Person 1's backend. Deployed on Vercel. |
| What you DON'T do | No backend code. No SQL. No embeddings. No Python. If a prompt seems to need any of those, the request belongs to Person 1 or Person 2. |
| Your Hour 0-48 path | Vite + Tailwind setup (Hours 0-2) → Mock data + design tokens (Hours 2-4) → Layout shell + dashboard (Hours 4-10) → Time Machine slider (Hours 10-14, demo centerpiece) → Search + chunk grid (Hours 14-20) → Discovery card (Hours 20-24) → Quiz UI (Hours 24-30) → Backend integration (Hours 30-36) → Deploy + polish (Hours 36-48). |
| Demo-critical task | The Time Machine slider must feel instant (under 100ms response). Hours 10-14 are carved out specifically for this. If it's laggy, the whole demo loses its 'wow' moment. |
| Branch strategy | Work on branch feat/frontend. You can develop standalone for the first 12 hours using mock JSON files — no backend required. Around Hour 30 you'll switch from mocks to Person 1's API. |

# Contents

# 1. Before You Start — Checklist

Verify each before the hackathon starts. Frontend tooling is well-behaved compared to Python, but Node version mismatches and npm install hangs do happen at venues with bad WiFi.

| ✓ / ✗ | Item | How to verify |
|---|---|---|
| | Node.js 20+ installed | Terminal: node --version (must be v20 or v22) |
| | npm or pnpm installed (pnpm preferred — faster) | Terminal: pnpm --version or npm --version |
| | Git installed and configured | git config --list |
| | GitHub access to team repo | git push to a test branch |
| | Code editor with LLM (Cursor/Copilot/etc.) | Open project, paste a test prompt |
| | Vercel account created (no project yet) | vercel.com — sign up with GitHub |
| | Cloned the team repo locally | git clone <repo> |
| | Created your branch | git checkout -b feat/frontend |
| | Read this entire document end to end | Don't write code until you have |
| | Confirmed JSON contract with Person 1 | See Section 2 — your TypeScript types must match exactly |

# 2. The Three-Person Integration Contract

## The Three-Person Integration Contract

All three of you must agree on these JSON shapes BEFORE any of you start building. If any field changes during the hackathon, the person changing it must notify the other two in Slack/Discord immediately. The contract is the single source of truth — everything else flows from it.

| Endpoint | Method | Owner | Returns |
|---|---|---|---|
| /api/health | GET | P1 routes, P2 logic | {categories: [{name, retention, urgency, count}]} |
| /api/health?time_offset_hours=N | GET | P1 routes, P2 logic | Same shape, computed at t+N |
| /api/ingest | POST | P1 fully | {status, chunks_count, doc_id} |
| /api/search | POST | P1 routes, P2 logic | {results: [{chunk_id, content, score, retention}]} |
| /api/discovery | GET | P1 routes, P2 logic | {has_discovery, chunk?, reason?} |
| /api/review/{chunk_id} | POST | P1 fully | {status, new_access_count} |
| /api/quiz/start | POST | P1 routes, P2 logic | {quiz_id, questions: [{q, options, correct}]} |
| /api/quiz/submit | POST | P1 fully | {score, correct_count} |

### The Chunk JSON Shape

This is the canonical shape of a Chunk object. Person 1 stores it, Person 2 enriches and retrieves it, Person 3 displays it. If you need a field not listed here, propose the addition in chat before adding it.

```
{ "id": "chk_8a3f...", // UUID, generated by P1
"content": "...chunk text 200-500 chars...", // P1 splits source
"source_type": "file", // P1 sets at ingest
"source_name": "research_notes.md", // filename
"category": "technical" | "personal" | ..., // P2 sets via Groq
"created_at": "2026-04-25T10:00:00Z", // ISO 8601
"last_accessed": "2026-04-25T10:00:00Z", // updated on review
"access_count": 0, // increments on review
"stability_S": 7.0, // P2 decay parameter
"complexity_k": 1.0 // P2 decay parameter
}
```

## Your contract with Person 1 (Backend)

You consume Person 1's REST endpoints. They will be on localhost:8000 during development and on Render during the demo. You hardcode neither — use VITE_API_BASE_URL env var.

**Your TypeScript types must match the JSON shapes above EXACTLY.** If Person 1 changes a field, your build will break. To prevent surprise, check in with P1 at Hours 4, 18, and 30.

## Your visual contract with Person 2 (Intelligence)

You don't talk to Person 2 directly — only through the JSON contract. But the categories they generate must match what your CSS knows about:

- **Categories are exactly four:** technical, personal, reference, general. Tell P2 if you ever change this list.
- **Retention is in [0, 1].** You'll color the dashboard cells by retention — design a continuous color scale. Don't bin into 'good/bad' too early.
- **Quiz questions have exactly 4 options.** Your UI assumes 4 buttons.
- **Discovery responses can be either** {has_discovery: true, chunk: {...}, reason: '...'} **or** {has_discovery: false}. Handle both branches.

# 3. Pre-Hackathon Setup (Do This DAYS Before)

## 3.1 — Install Node 20+ and pnpm

> **STOP — HUMAN STEP REQUIRED**
>
> **Why:** Vite 5+ requires Node 18+. We're using 20+ for safety. Mismatched Node versions cause cryptic install errors at the venue.
>
> **Steps:**
>
> 1. Install via nvm (recommended): nvm install 20 && nvm use 20
> 2. Or install Node 20 directly from nodejs.org
> 3. Verify: node --version shows v20.x or v22.x
> 4. Install pnpm globally: npm install -g pnpm
> 5. Verify: pnpm --version
>
> **Why pnpm over npm:** pnpm is 2-3x faster on install and uses less disk. Worth the 30 seconds to install. If your team prefers npm, that's fine — just be consistent.

## 3.2 — Pre-warm the npm cache

> **STOP — HUMAN STEP REQUIRED**
>
> **Why:** The first pnpm install downloads ~200MB of packages. On venue WiFi this can hang for 10+ minutes. Pre-warming on home WiFi caches everything.
>
> **Steps:**
>
> 1. On stable home WiFi, create a test Vite + React + TS project somewhere temporary:
> pnpm create vite@latest test-cache --template react-ts
> 2. cd test-cache && pnpm install — wait for completion
> 3. pnpm add tailwindcss postcss autoprefixer recharts lucide-react (these are what we'll need)
> 4. pnpm install again to confirm cache works
> 5. Delete the test folder: cd .. && rm -rf test-cache
> 6. Your pnpm cache (~/.local/share/pnpm/store or similar) now has all the packages locally. Subsequent installs at the venue are nearly instant.

## 3.3 — Create Vercel account & warm it up

> **STOP — HUMAN STEP REQUIRED**
>
> **Steps:**
>
> 1. Go to **vercel.com**, sign up using your GitHub account
> 2. Connect Vercel to GitHub (it'll ask for permissions)
> 3. Don't create a project yet — you'll do that at Hour 36
> 4. Verify your email so the account is fully active
> 5. Note: Vercel free tier is generous — 100GB bandwidth, no cold starts. Unlike Render's backend, your frontend will not have a sleep problem.

## 3.4 — Sync with Person 1 on the API URL

Before the hackathon, message Person 1: *'I'll be hitting your endpoints at localhost:8000 during dev. When you deploy to Render, send me the production URL so I can set VITE_API_BASE_URL before my Vercel deploy. Confirming you'll have CORS open for localhost:5173 (Vite default).'*

## 4. Hours 0-2 — Vite + React + TypeScript + Tailwind

Goal: a runnable Vite dev server with Tailwind configured, project structure ready for components, environment variables wired up.

### Prompt 1 — Bootstrap the Vite project

**PROMPT TO PASTE INTO YOUR LLM**

```
Inside the existing dory-md/ repository, the frontend/ folder already exists
(created by Person 1 as an empty directory).

Bootstrap a Vite + React + TypeScript project inside it. From the project root:

1. Run: cd frontend && pnpm create vite@latest . --template react-ts
(the dot tells Vite to use the current directory, not create a new one)
2. Install dependencies: pnpm install
3. Add the packages we'll need:
pnpm add tailwindcss postcss autoprefixer
pnpm add recharts lucide-react clsx
pnpm add -D @types/node

4. Initialize Tailwind: pnpm exec tailwindcss init -p
5. Configure tailwind.config.js content paths: ['./index.html',
'./src/**/*.{js,jsx,ts,tsx}']
6. Add Tailwind directives at the top of src/index.css (replacing all default
styles):
@tailwind base;
@tailwind components;
@tailwind utilities;

7. In src/App.tsx, replace the default content with a simple test:
<h1 className='text-3xl font-bold text-purple-600'>Dory.md is alive</h1>

8. Run: pnpm dev

Verify the page loads at http://localhost:5173 and shows the purple heading. If
Tailwind is not applying, the most common cause is content paths in
tailwind.config.js — fix and retry. After confirming it works, give me the exact
pnpm dev command output so I can see the local URL.
```

### Prompt 2 — Project structure

**PROMPT TO PASTE INTO YOUR LLM**

```
Inside frontend/src/, create this directory structure:

src/
■■■ components/ # Reusable UI components
■ ■■■ layout/ # AppShell, Header, Footer
■ ■■■ dashboard/ # Knowledge Health treemap, Time Machine slider
■ ■■■ search/ # Search bar, results list
■ ■■■ chunks/ # Chunk card, chunk grid
■ ■■■ discovery/ # Discovery notification card
■ ■■■ quiz/ # Quiz UI components
■ ■■■ ui/ # Generic primitives (Button, Card, Modal)
■■■ pages/ # Top-level page components
■■■ lib/ # API client, utilities
■ ■■■ api.ts # API client functions
■ ■■■ types.ts # TypeScript types matching backend contract
■ ■■■ utils.ts # Shared helpers
■■■ data/ # Mock JSON files (used Hours 0-30)
■■■ styles/ # Tailwind extensions, custom CSS
■ ■■■ theme.ts # Color tokens, spacing
■■■ App.tsx # Top-level app
■■■ main.tsx # React entry point
■■■ index.css # Tailwind directives + base styles

Create empty placeholder files (.gitkeep is fine) so the structure exists in
git. Don't write component code yet.
```

### Prompt 3 — Environment variables

**PROMPT TO PASTE INTO YOUR LLM**

```
Set up environment variables for switching between mock data and the live
backend.

1. Create frontend/.env.example with:
VITE_API_BASE_URL=http://localhost:8000
VITE_USE_MOCKS=true

2. Create frontend/.env (gitignored, copy from example) with the same values

3. Add frontend/.env to the project's .gitignore (if Person 1's gitignore at the
root doesn't already cover it)

4. In src/lib/config.ts, expose the env vars typed:
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ??
'http://localhost:8000';
export const USE_MOCKS = import.meta.env.VITE_USE_MOCKS === 'true';

5. In src/vite-env.d.ts, declare the types so TypeScript doesn't complain:
interface ImportMetaEnv {
readonly VITE_API_BASE_URL: string;
readonly VITE_USE_MOCKS: string;
}
interface ImportMeta { readonly env: ImportMetaEnv; }
```

## Commit checkpoint

git checkout -b feat/frontend then git add -A && git commit -m 'feat(frontend): scaffold Vite + React + TS + Tailwind' then git push -u origin feat/frontend. Open a draft PR.

## 5. Hours 2-4 — Mock Data + Design Tokens (Cosmic Theme)

Goal: TypeScript types matching the backend contract, mock JSON files realistic enough to design against, and the cosmic blue/purple theme that matches the GitHub README aesthetic.

### Prompt 4a — TypeScript types (part 1: chunk + dashboard)

**PROMPT TO PASTE INTO YOUR LLM**

```
In src/lib/types.ts, define TypeScript types matching the backend contract
EXACTLY. We'll do this in two prompts — paste this first one, then Prompt 4b
adds the rest.

// Categories — must match backend's Groq classifier output
export type Category = 'technical' | 'personal' | 'reference' | 'general';

// Urgency — derived from retention
export type Urgency = 'low' | 'medium' | 'high';

// A chunk as returned from the API
export interface Chunk {
id: string;
content: string;
source_type: 'file';
source_name: string;
category: Category;
created_at: string; // ISO 8601
last_accessed: string; // ISO 8601
access_count: number;
stability_S: number;
complexity_k: number;
}

// Dashboard health response
export interface CategoryStat {
name: Category;
avg_retention: number; // [0, 1]
count: number;
urgency: Urgency;
}

export interface HealthResponse {
categories: CategoryStat[];
total_chunks: number;
time_offset_hours: number;
}

After pasting, confirm src/lib/types.ts compiles cleanly with no TS errors. Then
move to Prompt 4b.
```

### Prompt 4b — TypeScript types (part 2: search + discovery + quiz)

**PROMPT TO PASTE INTO YOUR LLM**

```
Append these additional types to src/lib/types.ts (don't replace the file — add
to it):

// Search response
export interface SearchResult {
chunk_id: string;
content: string;
score: number;
retention: number;
source_name: string;
category: Category;
}

export interface SearchResponse {
results: SearchResult[];
}

// Discovery response — discriminated union, handle both branches
export type DiscoveryResponse =
| { has_discovery: true; chunk: Chunk; reason: string }
| { has_discovery: false };

// Quiz types
export interface QuizQuestion {
question: string;
options: [string, string, string, string]; // exactly 4
}

export interface QuizStartResponse {
quiz_id: string;
questions: QuizQuestion[];
}

export interface QuizSubmitResponse {
score: number;
correct_count: number;
detail: { question_id: number; was_correct: boolean; correct_index: number }[];
}

Match every field exactly — these are the contracts. Confirm the file compiles
with no TypeScript errors.
```

### Prompt 5 — Mock JSON files

**PROMPT TO PASTE INTO YOUR LLM**

```
In src/data/, create these mock JSON files that match the contracts from src/lib/types.ts.
They'll let me develop the entire UI before the backend is ready.

1. src/data/mock_health.json — a HealthResponse with realistic data:
• 4 categories with varying retention values: technical=0.65, personal=0.85, reference=0.45,
general=0.72
• Counts: technical=412, personal=234, reference=189, general=1465
• urgency derived from retention (>=0.7=low, 0.4-0.7=medium, <0.4=high)
• total_chunks=2300, time_offset_hours=0

2. src/data/mock_chunks.json — array of 30 Chunk objects with realistic content. Use diverse
source_names like 'research_notes.md', 'meeting_2025_03_14.md', 'react_patterns.md',
```

```
'morning_pages.md'. Mix categories (40% technical, 30% personal, 15% reference, 15%
general). Use realistic timestamps spanning the past 30 days. last_accessed for some chunks
should be older than 7 days (to look like 'forgotten').

3. src/data/mock_search_results.json — a SearchResponse with 8 SearchResult items, scores
between 0.4-0.95, retentions between 0.2-0.9.

4. src/data/mock_discovery.json — a DiscoveryResponse where has_discovery=true. The chunk
should be one with low retention. reason='You wrote about this 12 days ago — likely 60%
forgotten.'

5. src/data/mock_quiz.json — a QuizStartResponse with 5 QuizQuestion objects on diverse
topics.

Make the content feel real, not Lorem Ipsum. For technical chunks, write about plausible
things like 'Python asyncio gather behavior on exception', 'React useEffect cleanup',
'PostgreSQL EXPLAIN ANALYZE'. For personal: 'coffee shop in Capitol Hill', 'workout
routine', 'books to read'. The realism makes the demo land.
```

### Prompt 6 — Cosmic theme tokens

**PROMPT TO PASTE INTO YOUR LLM**

```
Configure the cosmic blue/purple/coral aesthetic that matches Dory.md's branding (deep space
+ accent colors).

1. In tailwind.config.js, extend the theme with custom colors:
theme: {
extend: {
colors: {
'cosmos': {
950: '#0a0e1a', // deepest space
900: '#0f172a', // base background
800: '#1e293b', // card background
700: '#334155', // muted borders
},
'nebula': {
500: '#7c3aed', // primary accent (purple)
400: '#a78bfa',
300: '#c4b5fd',
},
'pulsar': {
500: '#0891b2', // secondary accent (teal)
400: '#22d3ee',
},
'flare': {
500: '#f97316', // warning/discovery accent (coral)
400: '#fb923c',
},
},
fontFamily: {
mono: ['ui-monospace', 'JetBrains Mono', 'Menlo', 'monospace'],
},
}
}

2. In src/index.css, set the body background to cosmos-900 with a subtle radial gradient:
body { @apply bg-cosmos-900 text-slate-100 min-h-screen; }
body::before {
content: '';
```

```
position: fixed; inset: 0; z-index: -1;
background: radial-gradient(ellipse at top, #1e1b4b 0%, transparent 50%),
radial-gradient(ellipse at bottom right, #0e7490 0%, transparent 50%);
opacity: 0.3;
}

3. In src/styles/theme.ts, export a function for retention → color mapping that other
components will use:
export function retentionToColor(retention: number): string {
// 1.0 = strong nebula purple, 0.5 = pulsar teal, 0.0 = flare coral
// Interpolate cleanly so the dashboard treemap looks like a smooth gradient
if (retention >= 0.7) return '#7c3aed';
if (retention >= 0.5) return '#0891b2';
if (retention >= 0.3) return '#f97316';
return '#dc2626'; // critical, near-forgotten
}

4. Update App.tsx to test the theme:
<div className='p-8'>
<h1 className='text-4xl font-bold text-nebula-300'>Dory.md</h1>
<p className='text-pulsar-400 mt-2'>The notes app that remembers so you don't have to
forget.</p>
</div>

Verify the page looks like a deep-space dashboard, not a default white React app.
```

## Commit checkpoint

git commit -m 'feat(frontend): mock data + cosmic theme tokens'. At Hour 4, the visual identity is locked in. The rest of the build is just composing components inside this aesthetic.

## 6. Hours 4-10 — Layout Shell + Knowledge Health Dashboard

Goal: an AppShell with header/sidebar/main, and the Knowledge Health Dashboard rendering as a Recharts treemap from mock_health.json.

### Prompt 7 — Mock-aware API client

**PROMPT TO PASTE INTO YOUR LLM**

```
In src/lib/api.ts, build an API client that uses mock JSON files when VITE_USE_MOCKS=true,
otherwise hits the real backend.

import { USE_MOCKS, API_BASE_URL } from './config';
import mockHealth from '../data/mock_health.json';
import mockChunks from '../data/mock_chunks.json';
import mockSearch from '../data/mock_search_results.json';
import mockDiscovery from '../data/mock_discovery.json';
import mockQuiz from '../data/mock_quiz.json';
import { HealthResponse, SearchResponse, DiscoveryResponse, QuizStartResponse,
QuizSubmitResponse } from './types';

// Helper to add realistic latency in mock mode (so transitions look real)
const mockDelay = (ms = 200) => new Promise(resolve => setTimeout(resolve, ms));

export async function getHealth(timeOffsetHours: number = 0): Promise<HealthResponse> {
if (USE_MOCKS) {
await mockDelay(50); // make slider feel snappy in mocks
// Adjust mock retentions based on time offset to simulate decay
const adjusted = JSON.parse(JSON.stringify(mockHealth));
adjusted.time_offset_hours = timeOffsetHours;
adjusted.categories = adjusted.categories.map((c: any) => ({
...c,
avg_retention: Math.max(0, Math.min(1, c.avg_retention - 0.0005 * timeOffsetHours)),
}));
return adjusted;
}
const url = `${API_BASE_URL}/api/health?time_offset_hours=${timeOffsetHours}`;
const res = await fetch(url);
if (!res.ok) throw new Error(`Health API error: ${res.status}`);
return res.json();
}

Implement equivalent functions for: search(query, topK), getDiscovery(), startQuiz(),
submitQuiz(quizId, answers), reviewChunk(chunkId), getAllChunks(). Each follows the same
pattern: if mocks, return the JSON (with realistic delay); otherwise, fetch from
API_BASE_URL.

Throughout: throw errors on non-2xx responses. Components will handle them with React error
boundaries.
```

### Prompt 8 — AppShell layout

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/layout/AppShell.tsx — the persistent app frame.

Structure:
<div className='min-h-screen flex flex-col'>
<Header />
<div className='flex flex-1'>
<Sidebar />
```

```
<main className='flex-1 p-6 overflow-auto'>{children}</main>
</div>
</div>

Header (src/components/layout/Header.tsx):
• Left: 'Dory.md' wordmark in nebula-300, with the tagline 'remembers so you don't forget' in slate-400 to its right.
• Right: connection status indicator (a small green dot if backend reachable, gray if mocks). Show 'mocks' or 'live' label.
• Use a thin border-bottom in cosmos-700 for visual separation.
• Bg: cosmos-800 with backdrop-blur to feel slightly translucent against the gradient.

Sidebar (src/components/layout/Sidebar.tsx):
• 240px wide, full-height, cosmos-800 bg, border-right cosmos-700.
• Vertical list of nav items: Dashboard (Home icon), Search (Search icon), Quiz (BookOpen icon), Settings (Cog icon). Use lucide-react icons.
• Active nav item gets nebula-500 background, white text. Others get hover:bg-cosmos-700.
• At the bottom of the sidebar, show a small 'Last upload: 2 min ago' status (or 'No uploads yet') with a Database icon from lucide-react. This is a placeholder — Person 1's /api/stats endpoint is a stretch task that may or may not exist; render a static label for now.

App.tsx becomes:
<AppShell>
<Dashboard />
</AppShell>

Use React Router (pnpm add react-router-dom) to swap the main content area: / → Dashboard, /search → SearchPage, /quiz → QuizPage. For now Dashboard is the only one with content; the others can be placeholder 'Coming soon' divs.
```

### Prompt 9 — Knowledge Health treemap

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/dashboard/KnowledgeHealthTreemap.tsx — the centerpiece
visualization.

Recharts has a built-in Treemap component. Spec:

import { Treemap, ResponsiveContainer, Tooltip } from 'recharts';
import { CategoryStat } from '../../lib/types';
import { retentionToColor } from '../../styles/theme';

interface Props {
categories: CategoryStat[];
loading: boolean;
}

Each cell:
• Size proportional to category.count
• Fill color from retentionToColor(category.avg_retention)
• Label: category name + retention% (e.g., 'technical · 65%')
• On hover: show count, urgency, and a 'Click to filter' tooltip

The Treemap component takes data of shape:
[{ name: 'technical', size: 412, retention: 0.65, urgency: 'medium' }, ...]

Use a custom Treemap content prop to control rendering — Recharts' default
labels are ugly. Inside CustomCell, render an SVG <rect> with the right fill,
then a <text> with the label — clip if it doesn't fit.

Container: 100% width, 400px height, rounded-lg, bg-cosmos-800 with subtle
border.

Above the treemap, render a header:
<div className='flex items-baseline justify-between mb-4'>
<h2 className='text-xl font-semibold text-nebula-300'>Knowledge Health</h2>
<span className='text-sm text-slate-400'>{total_chunks} chunks across 4
categories</span>
</div>
```

### Prompt 10 — Dashboard page wiring it all together

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/pages/Dashboard.tsx that orchestrates the dashboard.

import { useState, useEffect } from 'react';
import { getHealth } from '../lib/api';
import { HealthResponse } from '../lib/types';
import KnowledgeHealthTreemap from
'../components/dashboard/KnowledgeHealthTreemap';

Behavior:
1. State: health (HealthResponse | null), loading (boolean), error (string |
null)
2. useEffect on mount: call getHealth(0), set state
3. Render:
• Loading state: skeleton placeholder for the treemap
• Error state: error message in flare-500
• Loaded: <KnowledgeHealthTreemap categories={health.categories} loading={false}
/>
4. Above the treemap, show 4 stat cards in a grid (one per category):
• Cat name (capitalized)
• Big retention percentage
• Small text 'X chunks'
• Urgency badge (green for low, amber for medium, red for high)

Layout:
<div className='space-y-6'>
<h1 className='text-3xl font-bold'>Your Knowledge, Right Now</h1>
<StatCardsRow categories={health.categories} />
<KnowledgeHealthTreemap ... />
{/* Time Machine slider goes here in the next section */}
</div>
```

### Prompt 11 — Smoke test the dashboard

**PROMPT TO PASTE INTO YOUR LLM**

```
Run pnpm dev and visit http://localhost:5173. Verify:
1. Dashboard loads (no console errors)
2. Treemap renders with 4 cells in different colors based on retention
3. Stat cards show realistic numbers and urgency badges
4. Background has the cosmic gradient
5. Header and sidebar are positioned correctly

If anything looks broken, paste the console errors and I'll help debug.
```

## Hour 10 milestone

git commit -m 'feat(frontend): dashboard with knowledge health treemap'. Take a screenshot and post it in team chat — Person 1 and Person 2 will be motivated by seeing the UI come together.

## 7. Hours 10-14 — Time Machine Slider

Goal: a horizontal slider that lets the user scrub forward in time, with the dashboard treemap and stat cards updating in real time as they drag. **This is the demo's signature moment.** If it lags, the demo loses its 'wow.'

### Performance requirements

- Drag should feel instant — under 100ms from drag to UI update
- No flicker between frames — Recharts handles this if data updates smoothly
- Debounce API calls so we don't fire 200 requests per drag
- Show the current time offset prominently so the user knows what they're seeing

#### Prompt 12 — Time Machine slider component

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/dashboard/TimeMachineSlider.tsx.

Props:
interface Props {
onTimeChange: (timeOffsetHours: number) => void;
}

Layout:
• Wrapper: rounded-lg, bg-cosmos-800, p-6, border border-cosmos-700
• Header row: 'Time Machine' title (left) + current time label (right, large, monospace)
- Time label format: 'now' if offset==0, '+X days' if 0 < offset < 168, '+X weeks' if 168 <=
offset < 720, '+X months' if >= 720. Use Math.round.
• Below: a custom slider input. Range: 0 to 4320 hours (= 6 months). Step: 1 hour.
• Below slider: tick labels at 'now', '+1 week', '+1 month', '+3 months', '+6 months'

Slider implementation:
• Use native <input type='range'> styled with Tailwind. Hide default thumb, draw custom with
::-webkit-slider-thumb and ::-moz-range-thumb pseudo-elements (this needs raw CSS in
src/index.css since Tailwind doesn't cover these).
• Track: gradient from nebula-500 (left, present) to flare-500 (right, future) — the color
hint reinforces 'forgetting increases over time'.
• Thumb: 20px circle, nebula-300 fill, white border, soft shadow.

State:
• const [offsetHours, setOffsetHours] = useState(0);
• const [isDragging, setIsDragging] = useState(false);

Handlers:
• onChange (during drag): setOffsetHours(value); throttled call to onTimeChange (debounce
50ms)
• onMouseDown: setIsDragging(true)
• onMouseUp / onTouchEnd: setIsDragging(false); call onTimeChange immediately with final
value

Throttling: use a ref to track the last call time. If less than 50ms since last call, use
setTimeout to call onTimeChange after the rest of the throttle window. This gives smooth
visual updates without spamming the API.

Add a small 'Reset to now' link below the slider that calls setOffsetHours(0) and
onTimeChange(0).
```

### Prompt 13 — Wire slider into Dashboard

**PROMPT TO PASTE INTO YOUR LLM**

```
Update src/pages/Dashboard.tsx to use the Time Machine slider.

1. Add state: const [timeOffset, setTimeOffset] = useState(0);

2. Change the useEffect to depend on timeOffset:
useEffect(() => {
getHealth(timeOffset).then(setHealth).catch(console.error);
}, [timeOffset]);

3. Add the slider above the treemap:
<TimeMachineSlider onTimeChange={setTimeOffset} />

4. Pass timeOffset to a small label inside the treemap header:
{timeOffset > 0 && (
<span className='text-flare-400 text-sm'>
Showing prediction at +{Math.round(timeOffset/24)} days
</span>
)}

5. Add a smooth transition class to the treemap cells so colors fade between
values instead of snapping. Recharts has an isAnimationActive prop on Treemap —
keep it true.
```

### Prompt 14 — Test the slider perceptibly

**PROMPT TO PASTE INTO YOUR LLM**

```
Open the dashboard. Drag the Time Machine slider slowly from left to right.
Verify:

1. The label updates in real time ('now' → '+1 day' → '+1 week' etc.)
2. The treemap cells visibly change color as you drag — getting more orange/red
toward the future
3. The stat cards' retention percentages decrease as time advances
4. There's no perceptible lag — drag feels glassy-smooth
5. Releasing the slider doesn't cause a jarring final state change

If the drag feels laggy, the most common cause is the API call firing too often.
Verify the throttle works — open the Network tab in DevTools and watch how many
/api/health requests fire during a slow drag. Should be ~5-10 max for a 1-second
drag, not 100.

Once the slider feels good, record a 5-second screen capture and post it in team
chat. This is the moment the team has been working toward — celebrate it.
```

**Heads-up**

**Performance tip:** if you have 2,000+ chunks in real data and the API is slow, the slider will feel less smooth in production than with mocks. Person 2 has optimized decay computation to under 1ms — the bottleneck is usually network roundtrip. If you see it being slow on the deployed app, increase the debounce to 100ms or use requestAnimationFrame to batch updates.

## Hour 14 milestone — major commit

git commit -m 'feat(frontend): time machine slider with realtime treemap updates'. Push the branch. The demo's signature moment now exists.

## 8. Hours 14-20 — Search + Chunk Grid

Goal: a search page where users can type a query, see ranked chunks, and click a chunk to mark it as 'reviewed' (which decreases its forgetting urgency).

### Prompt 15 — Chunk card component

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/chunks/ChunkCard.tsx — the visual building block for
chunks.

Props:
interface Props {
chunk: Chunk | SearchResult;
onReview?: (chunkId: string) => void;
showRetention?: boolean;
highlight?: string; // optional text to highlight (the search query)
}

Layout:
• Card: rounded-lg, bg-cosmos-800, border border-cosmos-700, p-4,
hover:border-nebula-500 transition. Cursor pointer if onReview is provided.
• Top row: source_name (small, slate-400) + category badge (right-aligned,
colored by retentionToColor inverse — bright if forgetting, muted if remembered)
+ retention indicator (small dot with retention percentage).
• Body: chunk.content. Truncate to 3 lines with line-clamp-3 unless expanded.
• If highlight prop is set, wrap matching text in <mark
className='bg-nebula-500/30 text-nebula-200'>...</mark>. Be careful with regex
escaping.
• Bottom row: 'Last accessed N days ago' (left) + Review button (right, only if
onReview provided)

The Review button: small, ghost style, label 'I read this'. On click, call
onReview(chunk.id). Show optimistic feedback — the card briefly flashes
nebula-300 then settles.

Date formatting: use a simple helper formatRelativeTime(iso: string) that
returns 'just now' (< 1 hour), 'X hours ago', 'X days ago', 'X weeks ago', 'X
months ago'. Don't pull in date-fns — write 30 lines of plain TS.
```

### Prompt 16 — Search bar component

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/search/SearchBar.tsx.

Props:
interface Props {
onSearch: (query: string) => void;
loading?: boolean;
}

Layout:
• Wrapper: relative position, full width, max-w-2xl, mx-auto.
• Input: large (h-14), rounded-xl, bg-cosmos-800, border border-cosmos-700,
focus:border-nebula-500, focus:ring-2 focus:ring-nebula-500/20, px-12 (room for
icon and clear button), text-lg, placeholder text-slate-500.
• Search icon (lucide Search) on the left, absolutely positioned.
• If query non-empty: X icon on right to clear.
• Placeholder text: 'Search your past self...'

Behavior:
• Debounce search calls by 300ms (use a ref + setTimeout, or useDebounce hook)
• Submit on Enter without waiting for debounce
• If loading prop is true, show a small spinner inside the input on the right
side

Below the input, when no search has been run yet, show 4-5 'try these' chip
buttons with sample queries: 'python async patterns', 'coffee shops', 'react
hooks', 'meeting notes'. Clicking a chip fills the input and submits.
```

### Prompt 17 — Search page

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/pages/SearchPage.tsx.

import { useState } from 'react';
import { search, reviewChunk } from '../lib/api';
import { SearchResult } from '../lib/types';
import SearchBar from '../components/search/SearchBar';
import ChunkCard from '../components/chunks/ChunkCard';

State:
• query: string
• results: SearchResult[] | null
• loading: boolean
• error: string | null

Behavior:
1. handleSearch(q: string): setQuery(q); setLoading(true); call search(q, 10), set results,
setLoading(false). Catch errors.
2. handleReview(chunkId: string): call reviewChunk(chunkId). On success, optimistically
increment access_count locally and bump retention to 1.0 in local state.

Render:
<div className='max-w-3xl mx-auto space-y-6'>
<div className='text-center space-y-2'>
<h1 className='text-3xl font-bold'>Find What You've Forgotten</h1>
<p className='text-slate-400'>Semantic + keyword search across everything you've written</p>
</div>
<SearchBar onSearch={handleSearch} loading={loading} />
```

```
{results && (
<div className='space-y-3'>
<p className='text-sm text-slate-400'>
Found {results.length} chunks matching "{query}"
</p>
{results.map(r => (
<ChunkCard
key={r.chunk_id}
chunk={...} /* convert SearchResult to Chunk-like shape */
onReview={handleReview}
highlight={query}
showRetention
/>
))}
</div>
)}
{results && results.length === 0 && (
<div className='text-center text-slate-400 py-12'>
No matches. Your past self may not have written about this yet.
</div>
)}
</div>
```

### Prompt 18 — Test search end-to-end

**PROMPT TO PASTE INTO YOUR LLM**

```
1. Navigate to /search
2. Click a 'try these' chip → results should populate from
mock_search_results.json
3. Type in the search bar → should debounce and re-fetch
4. Click 'I read this' on a result → should flash, then result's retention
indicator should jump to ~100%
5. Test with empty results state — temporarily edit mock_search_results.json to
have an empty results array, verify the 'No matches' message shows

If any step is broken, debug — paste the issue and I'll help.
```

## Commit checkpoint

git commit -m 'feat(frontend): search page with chunk cards and review action'

## 9. Hours 20-24 — Discovery Notification Card

Goal: a slide-in card that appears when the system finds a chunk worth re-surfacing. This is the second 'wow' moment of the demo — Dory.md actively reminds the user of something they're forgetting.

### Prompt 19 — Discovery card component

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/discovery/DiscoveryCard.tsx.

Props:
interface Props {
chunk: Chunk;
reason: string;
onDismiss: () => void;
onReview: (chunkId: string) => void;
}

Layout:
• Fixed position bottom-right: fixed bottom-6 right-6 z-50.
• Width: max-w-md, p-4, rounded-lg, bg-cosmos-800, border border-flare-500
(coral border because this is a 'noticed' moment).
• Shadow: shadow-2xl shadow-flare-500/20.
• Top row: small flare-500 dot + 'I just found something' (italic, slate-200) +
X dismiss button (right).
• Below: chunk content (3 lines max).
• Below: italicized reason in slate-400 (e.g., 'You wrote about this 12 days ago
— likely 60% forgotten').
• Bottom row: 'Read it' button (primary) + 'Later' link (secondary).

Animation:
• On mount: slide-in from right + fade-in. Use Tailwind's animate-in classes if
available, or define a custom keyframe in src/index.css:
@keyframes slide-in-right { from { transform: translateX(100%); opacity: 0; } to
{ transform: translateX(0); opacity: 1; } }
.discovery-enter { animation: slide-in-right 0.4s ease-out; }
• Add the class to the wrapper div on mount.

Behavior:
• 'Read it' button: calls onReview(chunk.id), then onDismiss
• 'Later' / X: just calls onDismiss
• Don't auto-dismiss — judges should see it and read it
```

### Prompt 20 — Discovery polling logic

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/lib/useDiscoveryPolling.ts — a custom hook that polls /api/discovery and shows a
card if has_discovery is true.

import { useState, useEffect } from 'react';
import { getDiscovery } from './api';
import { Chunk } from './types';

export function useDiscoveryPolling() {
const [discovery, setDiscovery] = useState<{chunk: Chunk; reason: string} | null>(null);

useEffect(() => {
let mounted = true;
let timeoutId: number;
```

```
const checkForDiscovery = async () => {
try {
const result = await getDiscovery();
if (!mounted) return;
if (result.has_discovery) {
setDiscovery({ chunk: result.chunk, reason: result.reason });
// Don't poll while card is showing — wait for dismiss
return;
}
// Poll again in 30 seconds
timeoutId = window.setTimeout(checkForDiscovery, 30000);
} catch (err) {
// Silently retry — discovery is best-effort
timeoutId = window.setTimeout(checkForDiscovery, 60000);
}
};

// First check after 5 seconds (so dashboard loads first)
timeoutId = window.setTimeout(checkForDiscovery, 5000);

return () => {
mounted = false;
window.clearTimeout(timeoutId);
};
}, []);

const dismiss = () => setDiscovery(null);

return { discovery, dismiss };
}

Then in App.tsx (or wherever AppShell lives), use this hook and render the DiscoveryCard if
discovery is non-null:

const { discovery, dismiss } = useDiscoveryPolling();
...
{discovery && (
<DiscoveryCard
chunk={discovery.chunk}
reason={discovery.reason}
onDismiss={dismiss}
onReview={(id) => { reviewChunk(id); dismiss(); }}
/>
)}
```

### Prompt 21 — Test the discovery flow

**PROMPT TO PASTE INTO YOUR LLM**

```
1. Reload the app. Wait 5 seconds.
2. The discovery card should slide in from the right with the mock chunk and
reason.
3. Verify the animation is smooth (no jank).
4. Click 'Later' — card disappears.
5. Wait 30 more seconds — card should reappear (mock returns has_discovery=true
again).
6. Click 'Read it' — card disappears AND the chunk's review is recorded (check
Network tab).

If you want to test with has_discovery=false, edit mock_discovery.json
temporarily — card should never appear and polling should silently retry every
30s.
```

## Commit checkpoint

git commit -m 'feat(frontend): discovery notification card with polling'

## 10. Hours 24-30 — Quiz Mode UI

Goal: a 5-question quiz flow with question display, answer selection, and a results screen showing score and per-question feedback.

### Prompt 22 — Quiz state machine

**PROMPT TO PASTE INTO YOUR LLM**

```
Quiz mode has clear states: idle → loading → question N/5 → submitting →
results. Build this as a finite state machine in src/pages/QuizPage.tsx.

type QuizState =
| { kind: 'idle' }
| { kind: 'loading' }
| { kind: 'playing'; quizId: string; questions: QuizQuestion[]; currentIndex:
number; answers: number[] }
| { kind: 'submitting' }
| { kind: 'results'; score: number; correctCount: number; detail: any[] }
| { kind: 'error'; message: string };

Transitions:
• idle → loading: user clicks 'Start Quiz'. Call startQuiz().
• loading → playing: API responded. Initialize answers to all -1.
• playing (Q1) → playing (Q2) ... etc. as user answers each question
• playing (Q5 answered) → submitting: call submitQuiz(quizId, answers)
• submitting → results: API responded
• results → idle: user clicks 'Take another'
• any → error: API failed

Use a useReducer or just a useState<QuizState> with a switch statement in the
render.
```

### Prompt 23 — Quiz UI components

**PROMPT TO PASTE INTO YOUR LLM**

```
Create src/components/quiz/ subcomponents:

1. QuizIntro.tsx — the idle state
• Heading: 'Quiz: How well do you remember?'
• Subhead: 'We'll generate 5 questions about chunks you might be forgetting'
• Big nebula-500 'Start Quiz' button

2. QuizQuestion.tsx — playing state
Props: { question: QuizQuestion; questionNumber: number; total: number;
selectedIndex: number | null; onSelect: (index: number) => void; onNext: () =>
void; }
Layout:
• Top: progress indicator '3 of 5' + a thin progress bar
• Question text: large, white
• 4 answer buttons stacked vertically:
- Default: bg-cosmos-800, border-cosmos-700, hover:border-nebula-500
- Selected: border-nebula-500, bg-nebula-500/10
- Each shows letter (A/B/C/D) + option text
• Next button at bottom: disabled if selectedIndex is null. Label: 'Next' for
Q1-Q4, 'Submit' for Q5.

3. QuizResults.tsx — results state
Props: { score: number; correctCount: number; detail: ...; onRetry: () => void;
}
Layout:
• Big circular score display: '4/5' in a circle, color-graded by score
• Below: 'You got 4 out of 5 correct. Great recall on...' (encouraging copy)
• A list of the 5 questions with checkmark/X for each, expandable to show what
the correct answer was for the ones you got wrong
• Buttons: 'Take another quiz' + 'Back to dashboard'
```

### Prompt 24 — QuizPage assembly

**PROMPT TO PASTE INTO YOUR LLM**

```
Wire it all together in src/pages/QuizPage.tsx using the state machine from
Prompt 22 and the components from Prompt 23.

Key behaviors:
1. On 'Start Quiz' click: dispatch loading, call startQuiz(), dispatch playing
2. On answer select: update answers[currentIndex] = selectedIndex (immutably).
Don't auto-advance.
3. On Next click: increment currentIndex. If it was Q5, dispatch submitting and
call submitQuiz.
4. On Submit results received: dispatch results.
5. On Retry click: dispatch idle.

Render based on state.kind with a switch statement, returning the appropriate
component.

Wrap the whole thing in <div className='max-w-2xl mx-auto'> and add gentle fade
transitions between states using a key prop on the wrapping div
(key={state.kind}-{currentIndex}).
```

### Prompt 25 — Test the quiz flow

**PROMPT TO PASTE INTO YOUR LLM**

```
1. Navigate to /quiz
2. Click 'Start Quiz'
3. Answer 5 questions in sequence
4. Verify the progress indicator updates ('3 of 5')
5. Try clicking Next without selecting an answer — should be disabled
6. After Q5 + Submit, verify results screen shows score, correct/incorrect
breakdown
7. Click 'Take another quiz' — should reset to idle

If any state transition is broken, walk me through the console error.
```

## Commit checkpoint

git commit -m 'feat(frontend): quiz mode with state machine'. Hour 30 — all major features built against mocks. Time to switch to real backend.

## 11. Hours 30-36 — Switch from Mocks to Real Backend

Goal: Person 1's backend at localhost:8000 is now running with real data from Person 2's intelligence modules. You flip VITE_USE_MOCKS to false and debug whatever breaks.

### Prompt 26 — Flip the mocks switch

**PROMPT TO PASTE INTO YOUR LLM**

```
Coordinate with Person 1: confirm their server is running on localhost:8000 with
the demo dataset loaded (Person 2 ran their seed_demo_database.py script).

Then in frontend/.env, set:
VITE_USE_MOCKS=false

Restart the Vite dev server (it doesn't auto-reload .env changes): pnpm dev

Open localhost:5173 and click through every page. Make a list of everything
that's broken — common issues:

• CORS errors in console — Person 1 needs to add localhost:5173 to allow_origins
• Missing fields in JSON responses — types don't match contract anymore
• Categories outside the expected 4 ('uncategorized' or empty string) — Person
2's classifier failed for some chunks; show a fallback color
• Slow API responses (1-3 seconds) — the first /api/health call rebuilds derived
data; subsequent calls should be fast

Paste any console errors and we'll fix them one at a time.
```

### Prompt 27 — Add error boundaries and graceful degradation

**PROMPT TO PASTE INTO YOUR LLM**

```
Real APIs fail in ways mocks never do. Add resilience.

1. Create src/components/ui/ErrorBoundary.tsx — a class component that catches
React errors. Falls back to a friendly message with a 'Reload' button.

2. Wrap each page in the boundary in App.tsx:
<Routes>
<Route path='/' element={<ErrorBoundary><Dashboard /></ErrorBoundary>} />
...
</Routes>

3. In api.ts, ensure every function has a try/catch and either returns a
sensible empty value or rethrows with a clearer message. Add a network-error
detection: if the fetch fails with TypeError (offline or backend down), throw
'Backend unreachable. Person 1 may be deploying.'

4. In each page, when state has error, render:
<div className='p-6 bg-cosmos-800 rounded-lg border border-red-500/40'>
<p className='text-red-400'>Couldn't load: {errorMessage}</p>
<button onClick={retry}>Retry</button>
</div>

Better to show a clean error than a white screen of doom during the demo.
```

### Prompt 28 — Loading states polish

**PROMPT TO PASTE INTO YOUR LLM**

```
With real API latency (50-300ms instead of mock 50ms), the loading states matter
more. Add subtle skeletons:

1. Dashboard: while health is loading, show 4 stat-card placeholders (gray
pulsing rectangles) and a treemap-shaped skeleton.

2. Search: while loading, show 3 ChunkCard skeletons with shimmer effect. Use
Tailwind's animate-pulse on a div with the same dimensions as the real card.

3. Quiz: when loading the quiz, show 'Generating quiz...' with a small spinner —
this can take 2-4 seconds because Groq is generating 5 questions.

Skeleton CSS pattern:
<div className='animate-pulse space-y-3'>
<div className='h-32 bg-cosmos-700 rounded-lg' />
<div className='h-4 bg-cosmos-700 rounded w-3/4' />
<div className='h-4 bg-cosmos-700 rounded w-1/2' />
</div>
```

## 12. Hours 36-44 — Deploy to Vercel

### STOP — HUMAN STEP REQUIRED

**Prerequisites:**

1. Person 1's backend is live at a Render URL (e.g., dory-backend.onrender.com)
2. Their backend's CORS allows your eventual Vercel URL — but you don't know the URL until after first deploy. **Workaround:** deploy with CORS open to '*' first, then tighten after you have the Vercel URL.
3. Your feat/frontend branch is pushed to GitHub

**Deploy steps:**

1. Go to **vercel.com/dashboard**
2. Click 'Add New' → 'Project'
3. Import the team GitHub repo. Vercel detects multiple folders.
4. Configure:
   - Framework Preset: Vite
   - Root Directory: **frontend** (CRITICAL — your code is in /frontend, not the repo root)
   - Build Command: pnpm build (or npm run build)
   - Output Directory: dist
   - Install Command: pnpm install
5. Environment Variables (add both):
   - VITE_API_BASE_URL = https://<person-1-render-url>
   - VITE_USE_MOCKS = false
6. Branch to deploy: feat/frontend (or main if you've merged)
7. Click 'Deploy'. First deploy takes 2-4 minutes.
8. **Once deployed**, copy the Vercel URL (e.g., dory-md.vercel.app)
9. Send the URL to Person 1 so they update their CORS settings on Render.
10. Send to the team: *'Frontend live at https://dory-md.vercel.app — backend at https://dory-backend.onrender.com — let's test the full deployed flow.'*

### Prompt 29 — Production smoke test

**PROMPT TO PASTE INTO YOUR LLM**

```
Open your Vercel URL in incognito mode (so cookies/localStorage don't mask
issues).

Verify in order:
1. Page loads without errors (open DevTools Console)
2. Dashboard shows real data (treemap renders, stat cards have real numbers)
3. Time Machine slider works smoothly with real backend
4. Search returns real results
5. Quiz starts and completes
6. Discovery card eventually appears (within 5-30s)
7. No CORS errors anywhere

If anything fails, paste the console error. Most production-only issues are:
• Missing env var (Vercel didn't pick it up)
• CORS (Person 1's Render service has stricter CORS than dev)
• Backend cold start (Render free tier sleeps; first request takes 30 seconds)

If the dashboard works, ping the team — production demo is live.
```

### Hour 44 milestone

git commit -m 'feat(frontend): production deployment to Vercel' (mostly config changes). Production app is live. Last 4 hours are polish + buffer.

## 13. Hours 44-48 — Polish, Bug Bash, Demo Prep

### Prompt 30 — Empty states and edge cases

**PROMPT TO PASTE INTO YOUR LLM**

```
Walk through these edge cases and make sure each has a clean UI state, not a
crash:

1. Backend returns 0 chunks (empty database): Dashboard should show 'No data yet
— upload files to get started' with a small icon. Treemap area shows an empty
silhouette.

2. Search returns 0 results: 'No matches found' with a sub-message 'Your past
self may not have written about this yet'.

3. Quiz with not enough chunks (less than 5): show 'Not enough content to quiz
on yet — upload more files first'.

4. Discovery returns has_discovery=false repeatedly: silently keep polling, no
UI element.

5. Backend completely unreachable: show a banner at the top 'Backend offline.
Showing cached data.' (in dev with mocks) or 'Backend unreachable' (in
production).

Test each by editing mock files temporarily or unplugging your network briefly.
```

### Prompt 31 — Demo polish details

**PROMPT TO PASTE INTO YOUR LLM**

```
Small touches that make the demo feel finished:

1. Add a favicon — use the cosmic-themed SVG from your GitHub README (download
it, save as frontend/public/favicon.svg, reference in index.html).

2. Update the page <title> in index.html: 'Dory.md — remembers so you don't
forget'.

3. Add a meta description for the share preview: 'Personal knowledge with a
memory'.

4. Add subtle hover states on every interactive element. Tailwind's hover:
utilities make this trivial. Skip none.

5. Add focus-visible styles for keyboard navigation: focus-visible:ring-2
focus-visible:ring-nebula-500 on buttons and inputs.

6. Verify the dashboard works on a 1920x1080 projector resolution (judges often
demo on a big screen). Open DevTools, set viewport to that size, check nothing
overflows.

7. Add a small attribution in the footer: 'Built at UWB Hacks 2026 by Persons 1,
2, 3' (replace with actual names).
```

## Hour 46-48: dry-run

With your team, do a 5-minute mock demo. Stand up, open the deployed site, narrate. Time it. The signature moments to land:

- Time Machine slider drag — show retention dropping over months
- Discovery card appearing mid-demo — pre-stage this if needed
- Search query that returns a forgotten chunk + click 'I read this'

## Hour 48: ship.

## 14. Stretch Tasks (if you finish early)

If you're done by Hour 30, ranked by demo impact:

- **Animated treemap transitions** — when the slider moves, cells should smoothly resize and recolor instead of snapping. Recharts has animation props but they're limited; consider Framer Motion.
- **Sound cue on discovery** — a soft chime when the card slides in. Subtle is the keyword.
- **Keyboard shortcuts** — / for search, Esc to close discovery, arrow keys for quiz answers (with letter A-D). Add a small '?' that shows the shortcuts.
- **Drag-and-drop file upload** — better than the current 'click to choose files' pattern.
- **Dark/light mode toggle** — the cosmic theme is dark; some judges prefer light. Toggle in the header.
- **Per-chunk timeline view** — click a chunk to see its access history as a line chart of retention over time.

## 15. Descope Ladder (if you fall behind)

Cut features in this order — top of the list goes FIRST:

| # | Cut | What you keep | Demo impact |
|---|---|---|---|
| 1 | Quiz mode UI entirely | All other pages | One screen less; demo focuses on dashboard + search |
| 2 | Discovery card | All other features | Less 'magical' but core demo intact |
| 3 | Search highlight in results | Search itself | Results still work, just less polished |
| 4 | Stat cards above treemap | Just the treemap | Slightly less context but treemap is the centerpiece |
| 5 | Vercel deploy | Localhost demo with screen share | Less impressive but still functional |

# 16. Handoff Checklists

## Hour 4 → tell P1: 'I have your TS types — confirm contract'

- Share src/lib/types.ts with P1 in chat
- Confirm Chunk shape matches their Pydantic schema
- If anything's different, agree on a single source of truth

## Hour 18 → tell P1: 'I'm done with the dashboard, what's the API ETA?'

- Confirm P1's endpoints are at least mock-shape-correct on localhost:8000
- Get a confirmation from P2 that demo data is loadable

## Hour 30 → 'Switching to real backend now'

- Pull both feat/backend and feat/intelligence into your local repo
- Set VITE_USE_MOCKS=false
- Make a list of broken things, work through them with P1 and P2

## Hour 36 → tell team: 'Vercel URL is X'

- P1 needs the URL to update Render's CORS allow_origins
- Everyone uses the URL for demo dry-runs

# 17. Troubleshooting Guide

| Symptom | Likely cause | Fix |
|---|---|---|
| pnpm install hangs forever | Slow venue WiFi, no cache | Use phone hotspot, or copy ~/.local/share/pnpm/store from a teammate |
| Tailwind classes don't apply | Wrong content paths in tailwind.config.js, or directives missing | Check tailwind.config.js content array; verify @tailwind directives in index.css |
| Vite dev server starts but page is blank | JS error in App.tsx | Open DevTools Console — usually a typo or missing import |
| CORS error in browser console | Backend doesn't allow your origin | Person 1 needs to add your URL (localhost:5173 or Vercel URL) to allow_origins |
| Treemap renders but cells are tiny squares | Recharts requires explicit width/height inside ResponsiveContainer | Wrap in a div with explicit height (e.g., h-96) |
| Time Machine slider lag | Too many API calls per drag | Increase debounce to 100ms; verify Network tab shows ~5-10 calls per drag |
| Vercel deploy fails on build | Mismatch between local and CI Node version, or missing env var | Check Vercel build logs; pin Node version in package.json's engines field |
| Production fetch returns 404 | VITE_API_BASE_URL not set or wrong | Check Vercel project's Environment Variables; rebuild after adding |
| Slow first load on production | Render free tier cold start (30s) | Add a useEffect that calls /api/keepalive on app mount; show loading state |
| Discovery card never appears | useDiscoveryPolling not running, or always returns has_discovery=false | Add console.log inside the hook; check mock_discovery.json |
| Quiz state machine stuck on 'submitting' | API call hangs or throws unhandled | Check Network tab; verify quiz_id is being sent correctly |
| Chunk content overflows card | Missing line-clamp | Add line-clamp-3 (or whatever count) and overflow-hidden |
| Mock and real backend disagree on field names | Drift since types.ts was first written | Re-sync with Person 1; whichever is wrong fixes it; don't paper over with type assertions |

*Person 3 Build Guide — Frontend*
*Last updated April 2026*
*If something is wrong or unclear, ask in team chat. Don't suffer alone.*